

Zadanie 1: Panel wniosków - tabela sterowana metadany

Senior Frontend Developer (React) | wersja 3.0

Pole	Wartość
Czas	maksymalnie 3 godziny
Obszar wiodący	UI sterowany metadany, tabela, modelowanie stanu, filtrowanie i sortowanie
Materiały wejściowe	/data/columns.json oraz /data/rows.json
Technologia	React + TypeScript
Oczekiwany wynik	kod źródłowy + README.md + skrypty dev/typecheck/test

1. Wstęp

To zadanie ma charakter praktyczny i symuluje realistyczny problem frontendowy. Nie oczekujemy rozbudowanej aplikacji produkcyjnej ani perfekcyjnej warstwy wizualnej. Ważniejsze są: poprawność działania, czytelność kodu, adekwatne decyzje techniczne, rozsądna priorytetyzacja oraz umiejętność utrzymania jakości w ograniczonym czasie.

2. Zasady realizacji

- Maksymalny czas pracy: 3 godziny. Po tym czasie zakończ pracę i opisz w README, co zrobił(a)byś dalej przy dodatkowych 60-90 minutach.
- Obowiązkowo: React + TypeScript. Dopuszczalny jest lekki setup projektu, np. Vite lub Next.js.
- Projekt powinien uruchamiać się lokalnie po standardowym npm install oraz npm run dev.
- Udostępnij skrypty: dev, typecheck i test. Testy nie muszą być liczne, ale powinny dotyczyć realnego zachowania.
- Dobór bibliotek jest dowolny i powinien być proporcjonalny do skali zadania. Zakres zadania opisuje zachowania, które rozwiązanie ma realnie obsługiwać.
- Nie trzeba budować realnego backendu. Wystarczy mock, lokalny adapter danych albo symulowany request na bazie dostarczonych plików.
- Korzystanie z zewnętrznych baz wiedzy oraz AI i agentów AI jest dozwolone.
- Dbaj o higienę pracy: nie wklejaj danych poufnych do narzędzi AI, weryfikuj wygenerowany kod i nie oddawaj fragmentów, których nie rozumiesz.

3. Kontekst zadania

Budujesz mały panel prezentujący listę wniosków. Backend dostarcza metadane kolumn oraz dane wierszy. UI ma reagować na metadane: kolejność, widoczność, etykiety, typy wartości i dostępność akcji powinny wynikać z danych wejściowych, a nie z twardo wpisanej tabeli.

4. Zakres do realizacji

- Wczytaj columns.json i rows.json z katalogu data albo użyj ich jako lokalnych fixture'ów.
- Zbuduj tabelę, której kolumny wynikają z metadanych: label, key, type, widoczność i kolejność.
- Obsłuż podstawowe typy wartości w czytelny sposób: tekst, liczba, data, kwota/status lub równoważne typy wynikające z danych.
- Dodaj sortowanie co najmniej dla tekstu, daty i wartości liczbowej. Pokaż sensowne zachowanie dla brakujących wartości.
- Dodaj filtrowanie po statusie oraz wyszukiwanie tekstowe po co najmniej jednym polu opisowym.
- Pokaż akcje wiersza zależnie od danych wiersza i/lub metadanych. Akcja niedostępna nie powinna wyglądać jak aktywna.
- Obsłuż stany: loading, success, empty oraz error.

8. Dodaj minimum 2 testy automatyczne: jeden dla logiki tabeli lub filtrowania, drugi dla renderowania/akcji albo sortowania.
9. W README opisz przyjęty model metadanych i decyzje techniczne.

5. Kryteria akceptacji

- tabela renderuje się na podstawie metadanych, a nie ręcznie wpisanej listy kolumn,
- sortowanie i filtrowanie działają dla reprezentatywnych przypadków,
- akcje wiersza respektują dane/warunki dostępności,
- obsłużono puste dane i błąd,
- minimum 2 testy przechodzą poprawnie,
- README pozwala uruchomić projekt i rozumieć główne decyzje.

6. Materiały wejściowe

Plik	Opis
columns.json	metadane kolumn i zachowania tabeli
rows.json	przykładowe rekordy wniosków

7. Oczekiwany sposób oddania

- kod źródłowy rozwiązania,
- README.md z instrukcją uruchomienia, listą założeń i krótkim opisem decyzji technicznych,
- w README: nazwanie zasad programowania, którymi kierowałeś/aś się w rozwiązaniu, z 1-2 przykładami w kodzie; nie narzucamy listy wzorcowej zasad,
- działające skrypty dev, typecheck i test,
- krótką sekcję README: Co zrobić(a)bym dalej przy dodatkowych 60-90 minutach.

8. Uwagi końcowe

Skup się na jakości i jasnym uzasadnieniu decyzji. Niepełne, ale kontrolowane rozwiązanie jest lepsze niż rozwiązanie pozornie kompletne, lecz nieczytelne albo trudne do utrzymania.